



UNIVERSITY OF TARTU



Enterprise System Integration (MTAT.03.229)

LECTURE 1: INTRODUCTION

**MOHAMAD GHARIB
UNIVERSITY OF TARTU**

Part I

Motivating scenario and problem statement

Enterprise system scenario

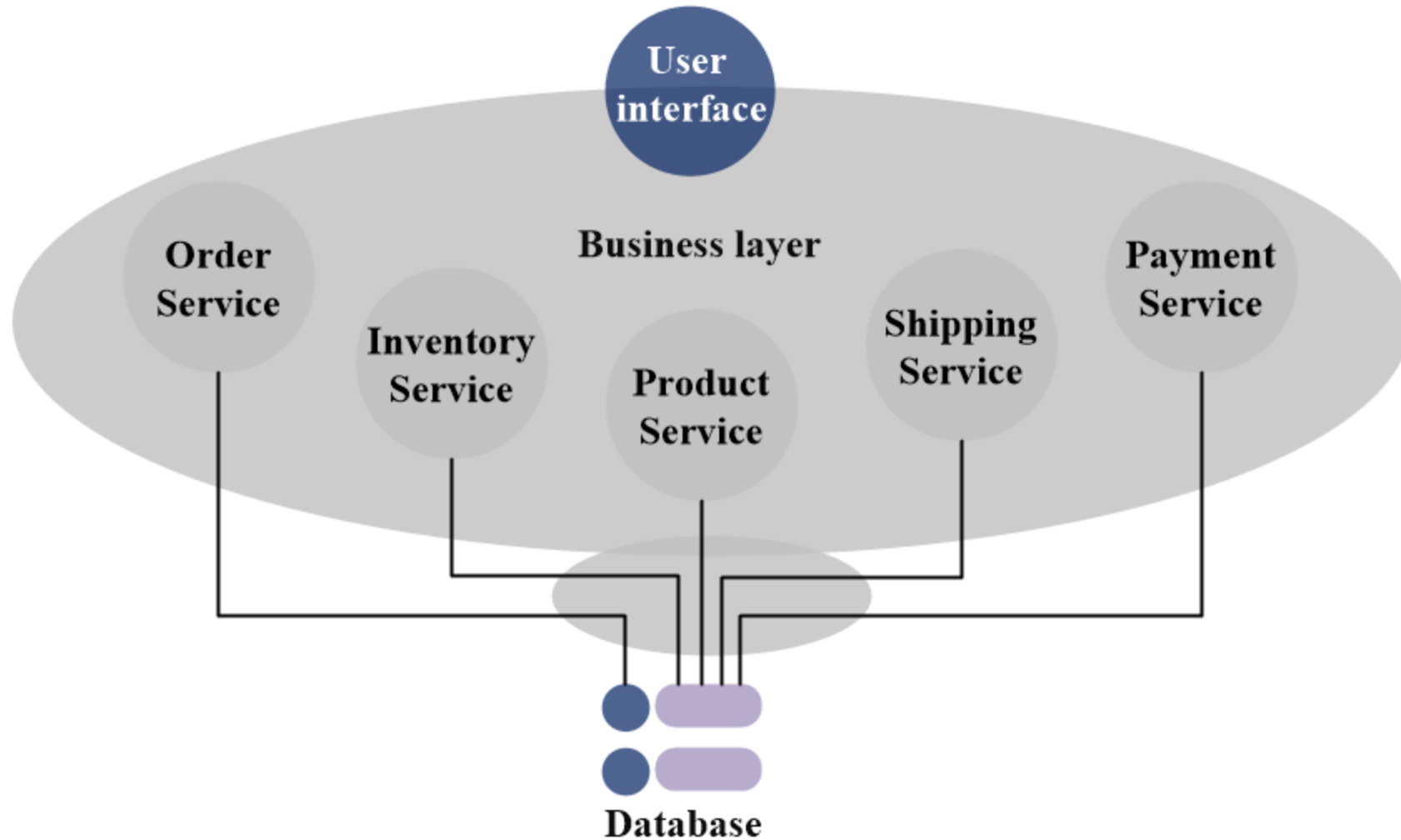
Information System (IS) of an online store – Let's consider five key functionalities of this system:

- **Products service (catalog)**: product presentation (item characteristic, price, availability, weight, delivery options, customer reviews, etc.).
- **Order service**: placing a purchase order for available product(s),
- **Inventory service**: how many items are still available, forecasting how many customers might order the item, forecasting item availability at a future date, etc.
- **Payment service**: available payment methods, verifying customer payment, etc.
- **Shipping service**: customer location, expected time required for the delivery, item dimensions, item weight, delivery cost, etc.

Can you see **dependencies** across functional areas?

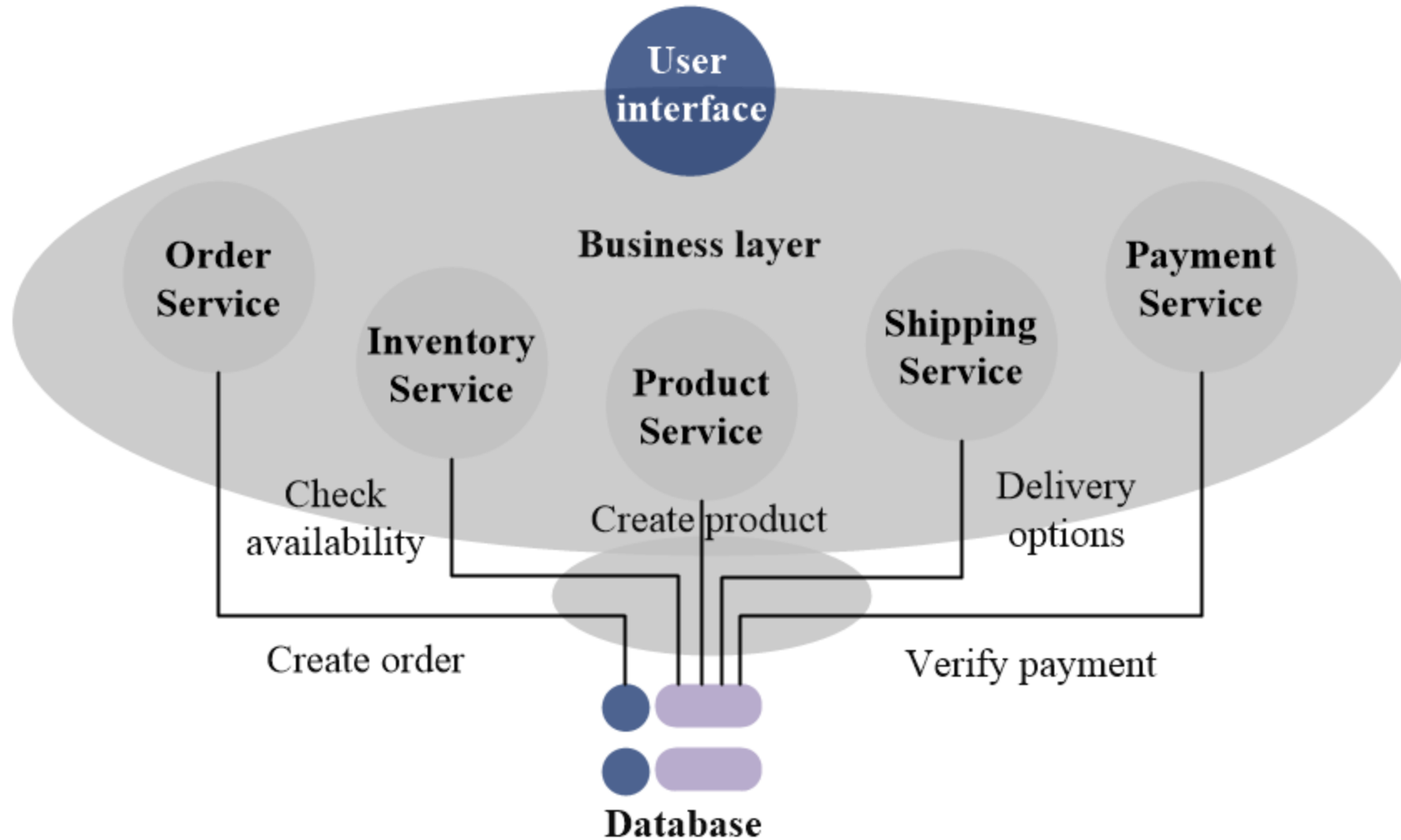
Enterprise system scenario

Monolithic/SOA architecture



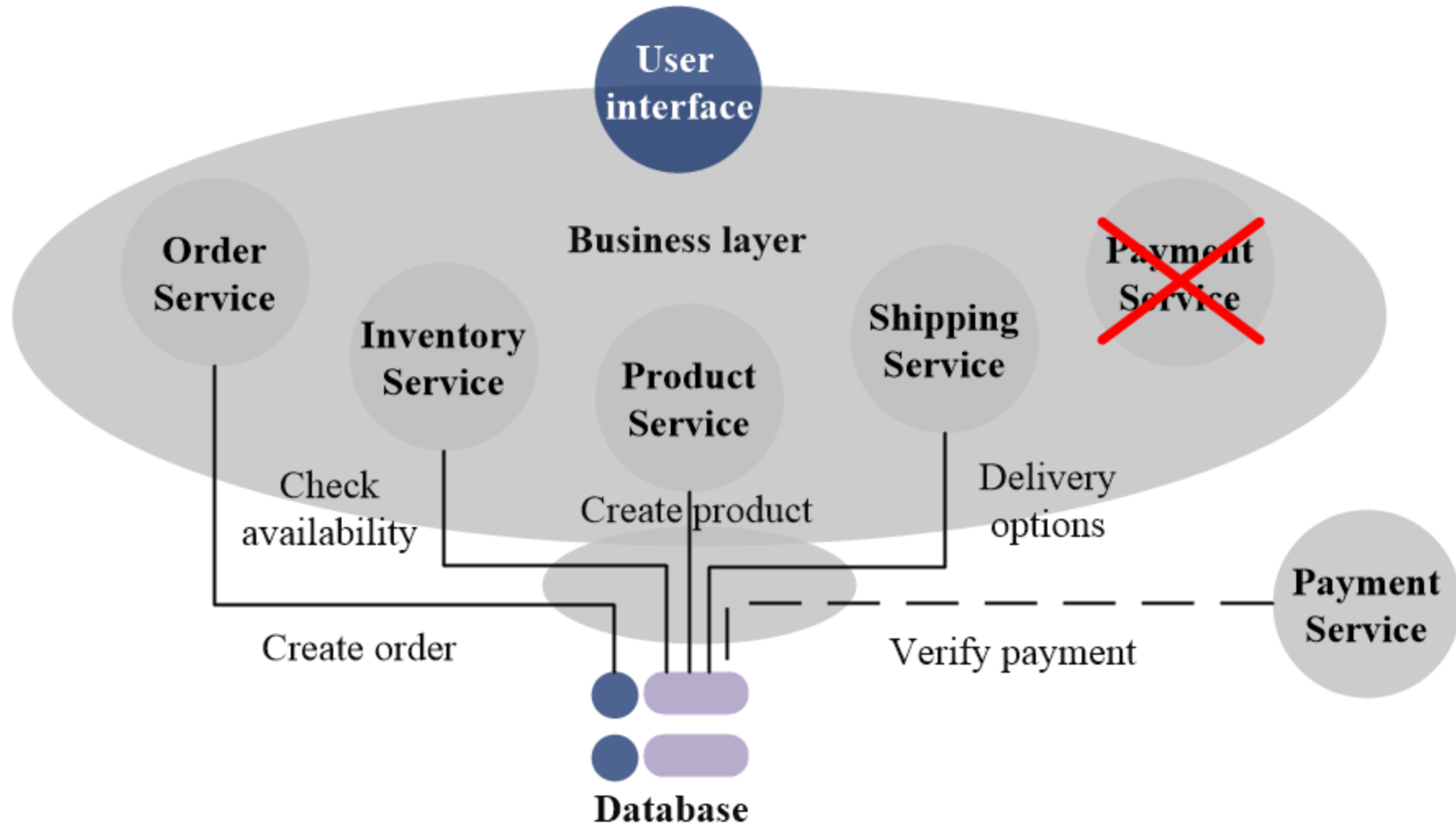
Enterprise system scenario

Monolithic/SOA architecture



Enterprise system scenario

Monolithic/SOA architecture





Enterprise system scenario

Consider that the online store wants to offer a **new service** that allow registered customers to gain points on purchases, and discounts on items/delivery for some items.

How we can implement this service?

Enterprise system scenario

Consider that the online store wants to offer a **new service** that allow registered customers to gain points on purchases, and discounts on items/delivery for some items.

How we can implement this service?

- A new **authentication service** that allows a customer to register in the service, which allows to differentiate registered and unregistered customers.
- A new **service** to keep track of the purchases of the registered customers, and calculate the extra points for any new purchase.
- A new **service** that allows transferring the points into discounts.
- A new **service** to encourage unregistered customers to register (e.g., offer them a discount on a purchase if they are registered).
- A new **service** to identify the items that we can offer a discount on for such customers.



Enterprise system scenario

Consider that the online store wants to offer a discount or free shipping for item(s) based on its/their price(s).

How we can implement this feature?

Enterprise system scenario

Consider that the online store wants to offer a discount or free shipping for item(s) based on its/their price(s).

How we can implement this feature?

- A new **service** to estimate potential delivery point.
- A new **service** to estimate the cost of the delivery.
- A new **service** to estimate the discount that can be offered.

Enterprise system scenario

Consider that the online store wants to offer a discount or free shipping for item(s) based on its/their price(s).

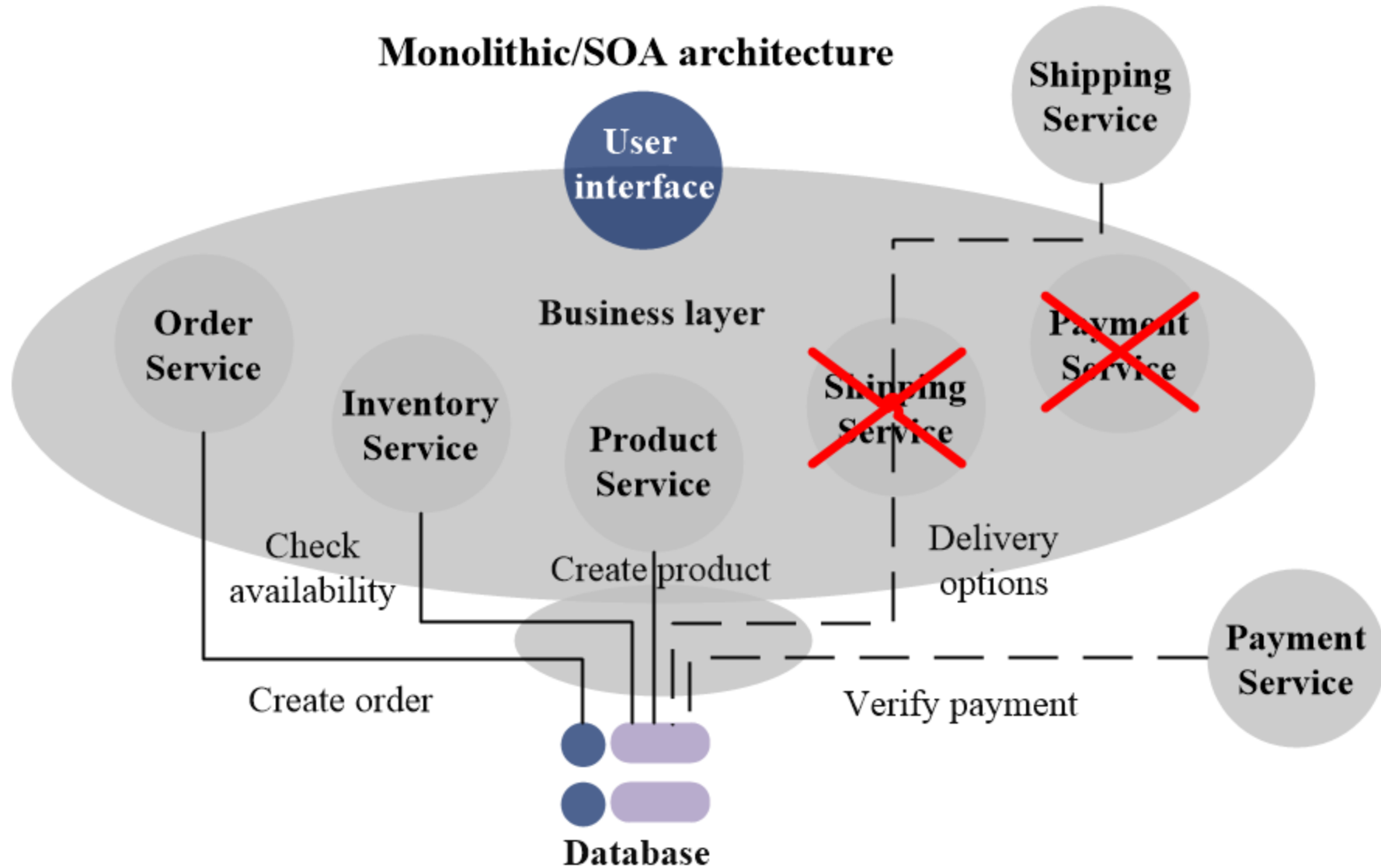
How we can implement this feature?

- A new **service** to estimate potential delivery point.
- A new **service** to estimate the cost of the delivery.
- A new **service** to estimate the discount that can be offered.

Consider that the online store wants to **offer a shipment tracking service.**

How we can implement this service?

Enterprise system scenario



Part II

Enterprise system integration and interoperability
- Basic concepts and definitions

ES integration and interoperability

An **enterprise** is one or more **organizations** sharing a **definite mission, goals** and **objectives** to offer an output such as a **product** or a **service**¹.

Enterprise integration is the **process** of ensuring the **interaction** between **enterprise entities** necessary to **achieve domain objectives**².

Interoperability is the ability for **two systems** to understand one another and to use functionality of one another³.

¹ISO 15704, Industrial Automation Systems—Requirements for Enterprise-reference Architectures and Methodologies, 2000.

²EN/ISO I9439, Enterprise Integration—Framework for Enterprise Modelling, 2003.

³D. Chen, F. Vernadat, Standards on enterprise integration and engineering—a state of the art, International Journal of Computer Integrated Manufacturing 17 (3) (2004) 235–253

ES integration and interoperability

Enterprise integration/interoperability can be approached at various levels⁴: (i) physical level (e.g., at the hardware level), (ii) application level (integration of software applications and database systems) and (iii) business level (co-ordination of functions that manage, control and monitor business processes).

Integration/interoperability is considered as **significant** if the interactions can take place at the three different levels.

⁴F.B. Vernadat, Enterprise Modeling and Integration: Principles and Applications, Chapman & Hall, London, 1996.



ES integration vs ES interoperability

Interoperability has the meaning of coexistence, autonomy and federated environment.

Integration refers more to the concepts of coordination, coherence and uniformization.

Does any of these concepts imply the other?

ES integration vs ES interoperability

Interoperability has the meaning of coexistence, autonomy and federated environment.

Integration refers more to the concepts of coordination, coherence and uniformization.

Does any of these concepts imply the other?

Two integrated systems are interoperable; but two interoperable systems are not necessarily integrated⁵

⁵D. Chen, D. Doumeingts, European Initiatives to develop interoperability of enterprise applications—basic concepts, framework and roadmap, Journal of Annual reviews in Control 27 (3) (2003) 151–160

Part III

Evolution of [enterprise] system architecture

Enterprise system architecture

An architecture is a description of the basic arrangement and connectivity of the different **parts of a system** (either a physical or a conceptual object or entity)¹.

An architecture has various meanings depending on its contextual usage⁶:

1. A **formal description** of a **system** at **component level** to guide its **implementation**;
2. The **structure of components**, their **interrelationships** and the principles and guidelines governing **their design**;
3. **Organizational structure** of a **system** or **component**.

¹ISO 15704, Industrial Automation Systems—Requirements for Enterprise-reference Architectures and Methodologies, 2000.

⁶Open Group, TOGAF: The Open Group Architecture Framework, Document No. 1910, Version 6, December 2000.

Enterprise system architecture

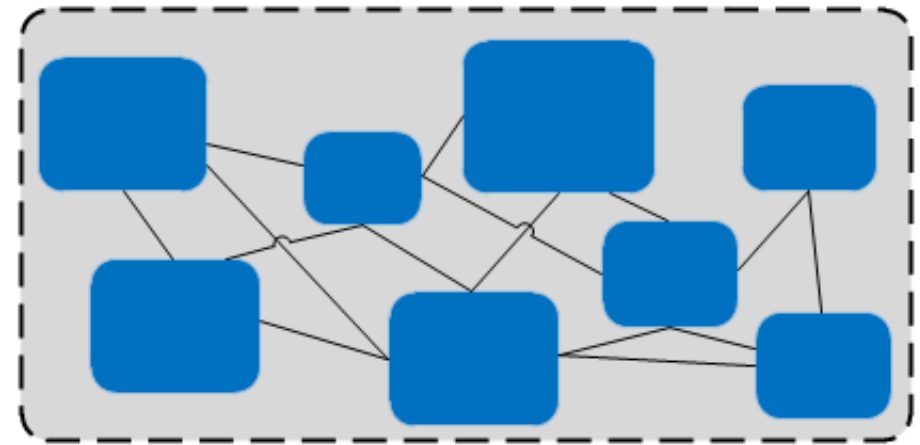
An **Enterprise Architecture (EA)** defines the **components** that make up the **overall system** and provides a **blueprint** from which the system can be **developed**.

Designing **enterprise systems** requires starting at **high-level abstractions** to ensure global view of the system that organize and represent enterprise systems at **sub-system** and **components levels**.

System



System Architecture



Enterprise System Architecture



Do we need an [enterprise] system architecture?

How to define Enterprise Architecture

The process of constructing an enterprise architecture involves,

- identify key **components (building blocks)** that compose the enterprise system;
- identify the **relationships** among these components;
- identify how these components use each other's functionality and data to achieve their own functionalities;
- identify how control is managed between the components.

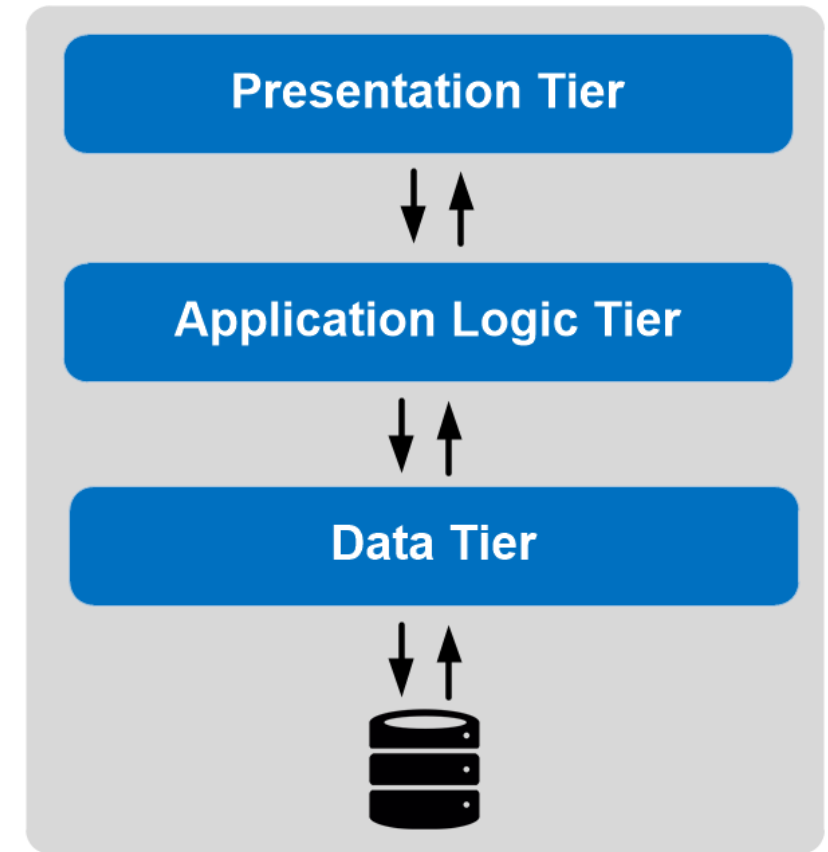
Types of [ES] Software Architecture:

Traditionally, a system can be decomposed into **three layers**:

Presentation layer (User Interface),
what the end-user interact with;

Application logic layer (Business),
where the reasoning/analysis take place;

Data layer: manages the access and manipulation of the data manipulated by the application logic.

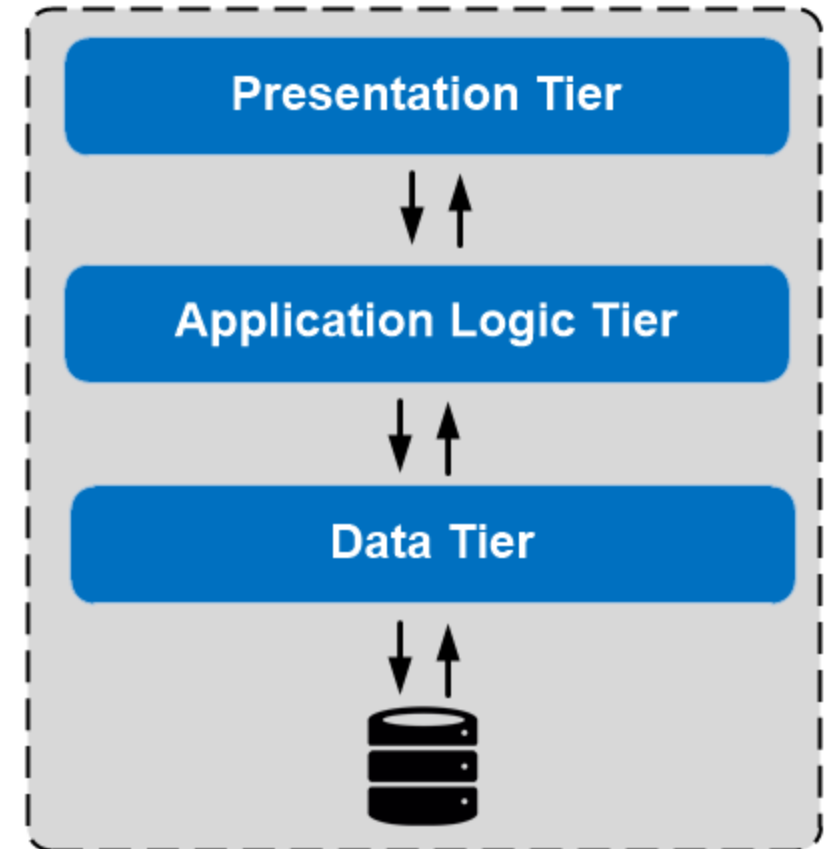


Types of [ES] Software Architecture:

[ES] Software Architecture can be:
One, Two, Three, or N-Tier.

One-tier architecture

- All layers are bundled in a monolithic entity.
- Typical “mainframe” architecture
 - Users access the system through terminals.
 - All computation happens in a single computer/platform.

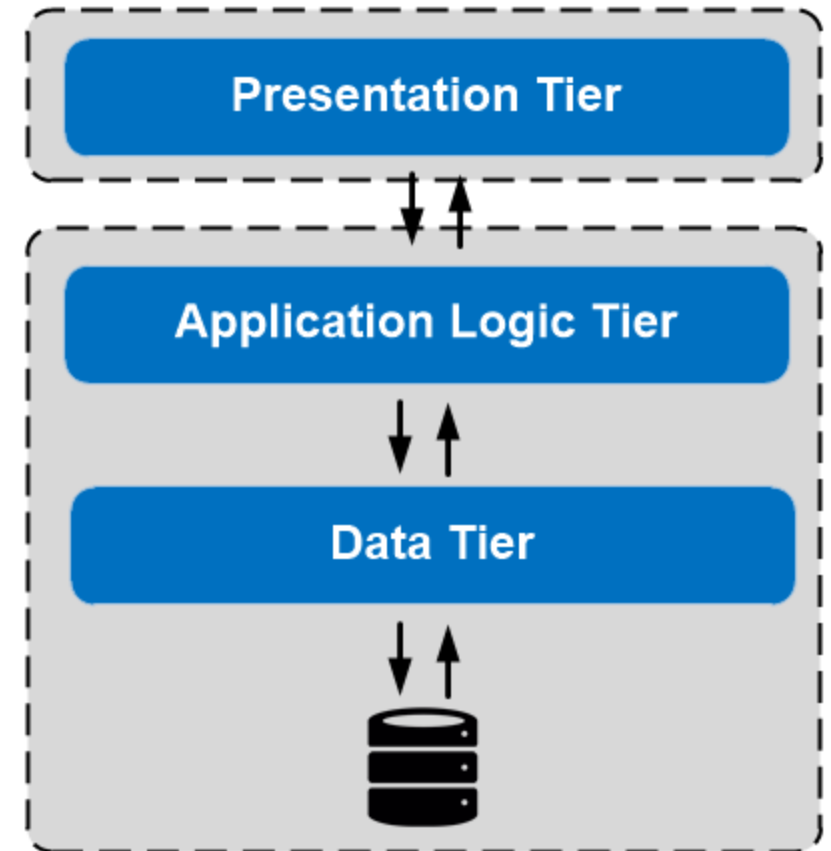


Types of [ES] Software Architecture:

[ES] Software Architecture can be:
One, Two, Three, or N-Tier.

Two-tier architecture

- Two-tier architecture is the original **client-server architecture**, consisting of a presentation tier and application and data tier.
- **Application layer(s)** can be **distributed** depending on the computing power of **clients**.
- The concept of **API** makes its appearance.

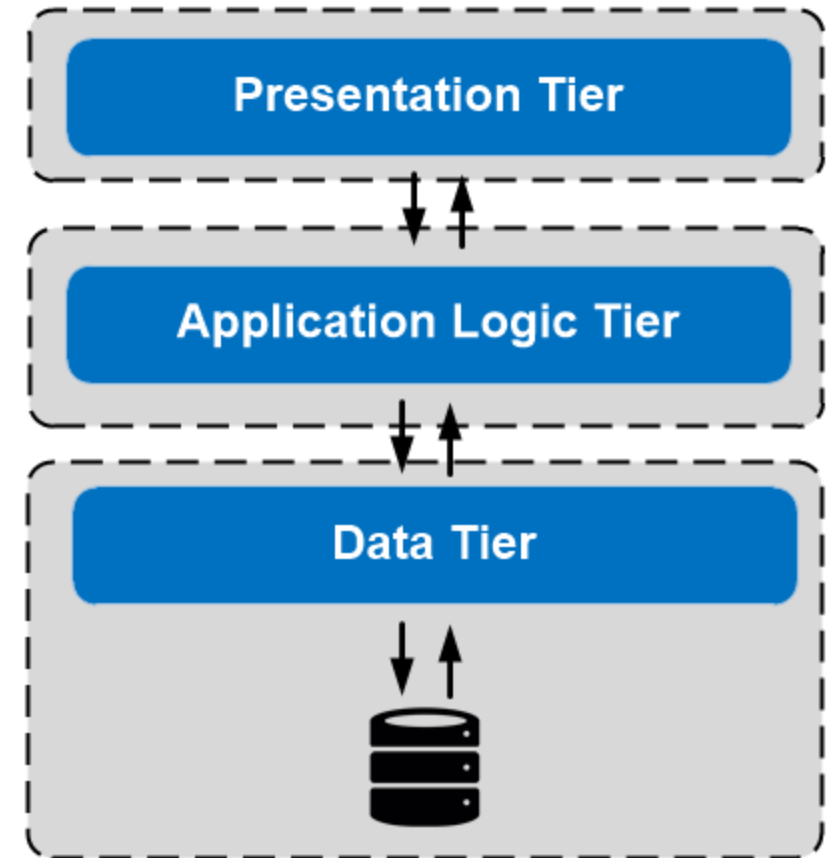


Types of [ES] Software Architecture:

[ES] Software Architecture can be:
One, Two, Three, or N-Tier.

Three-tier architecture

- Three-tier architecture is a well-established software application architecture that organizes applications into three logical and physical computing tiers: the presentation tier, or user interface; the application tier, where data is processed; and the data tier, where the data associated with the application is stored and managed.



Part IV

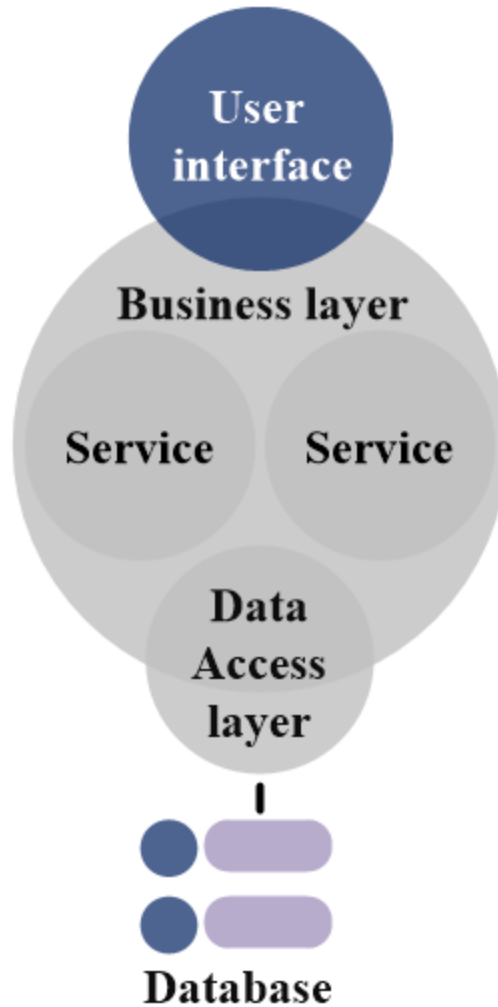
Monolithic vs Service-oriented architecture (SOA) vs Microservices Architectures

Monolithic vs SOA vs Mservices arch.

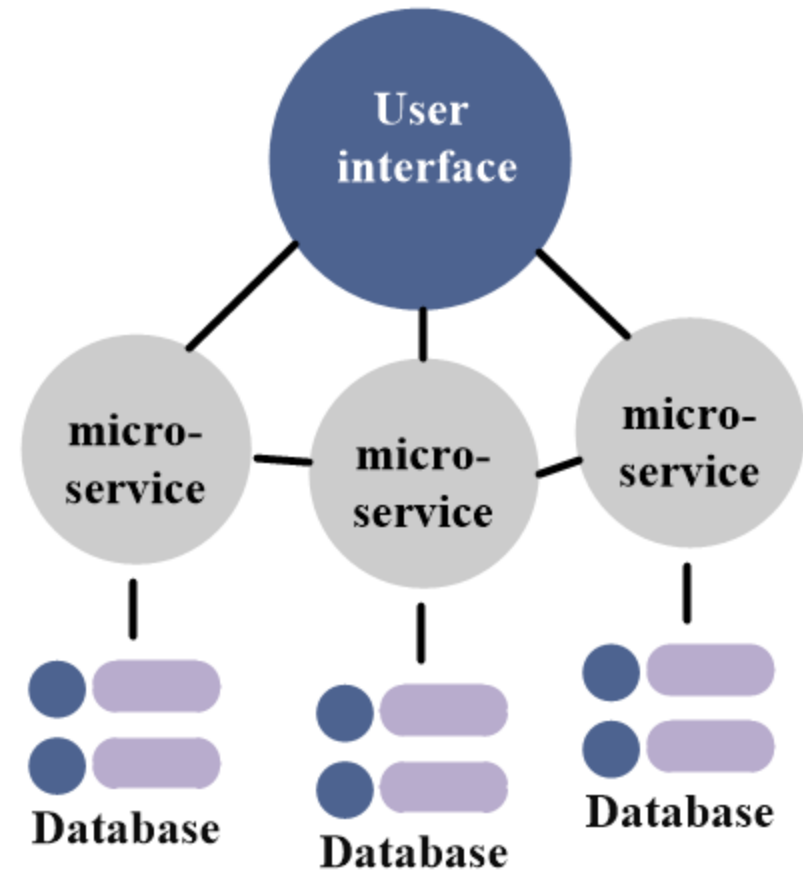
Monolithic architecture



SOA architecture

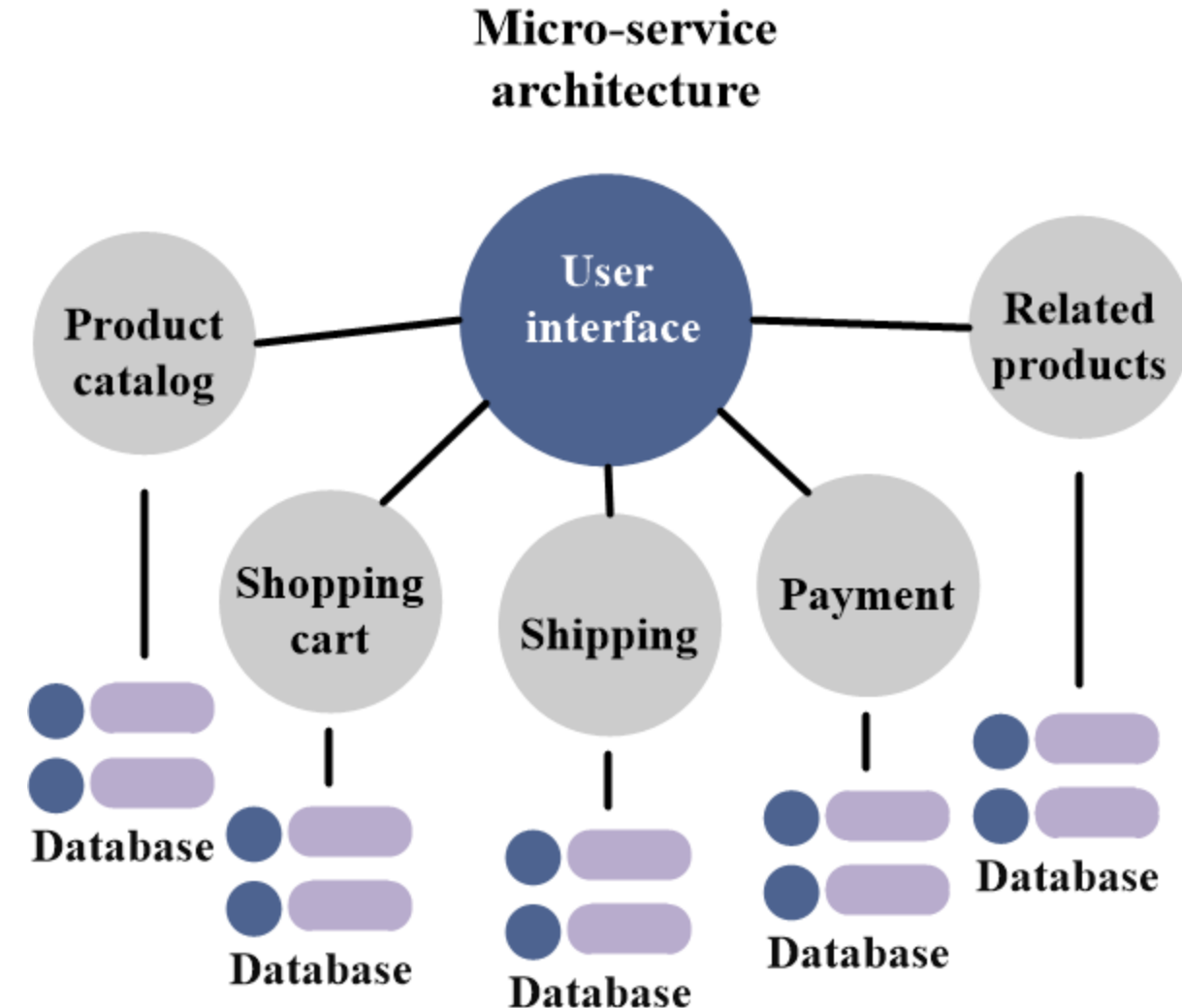


Micro-service architecture



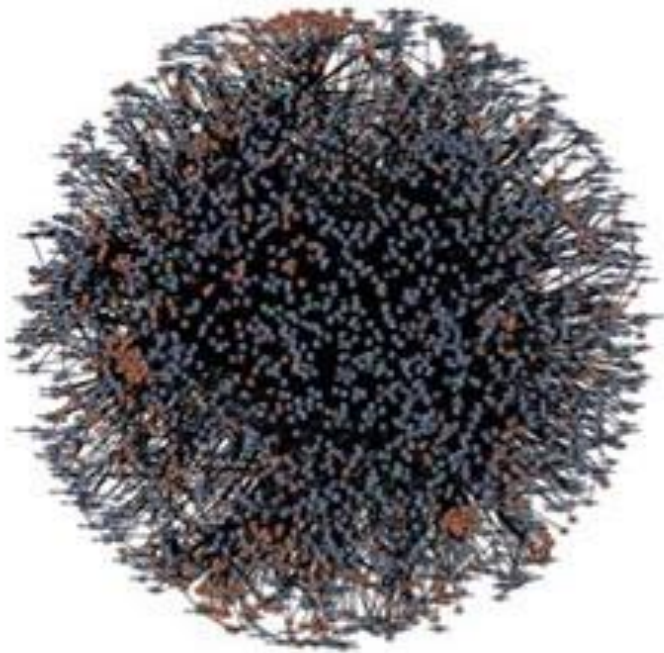
Microservices

- Unlike **monolithic architecture**, which consists of “one” entity, **microservices architecture** is created by cooperating services.
- **Microservices** were born out of **SOA** (Service-Oriented Architecture), as a solution to the challenges associated with monolithic applications.
- **Microservices** is effectively pushing monolithic architecture out of the market.

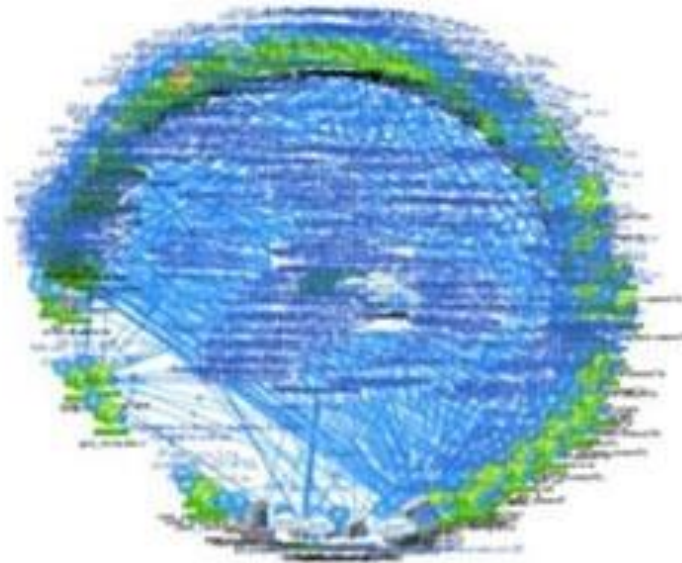


Microservices

500+ Microservices



amazon

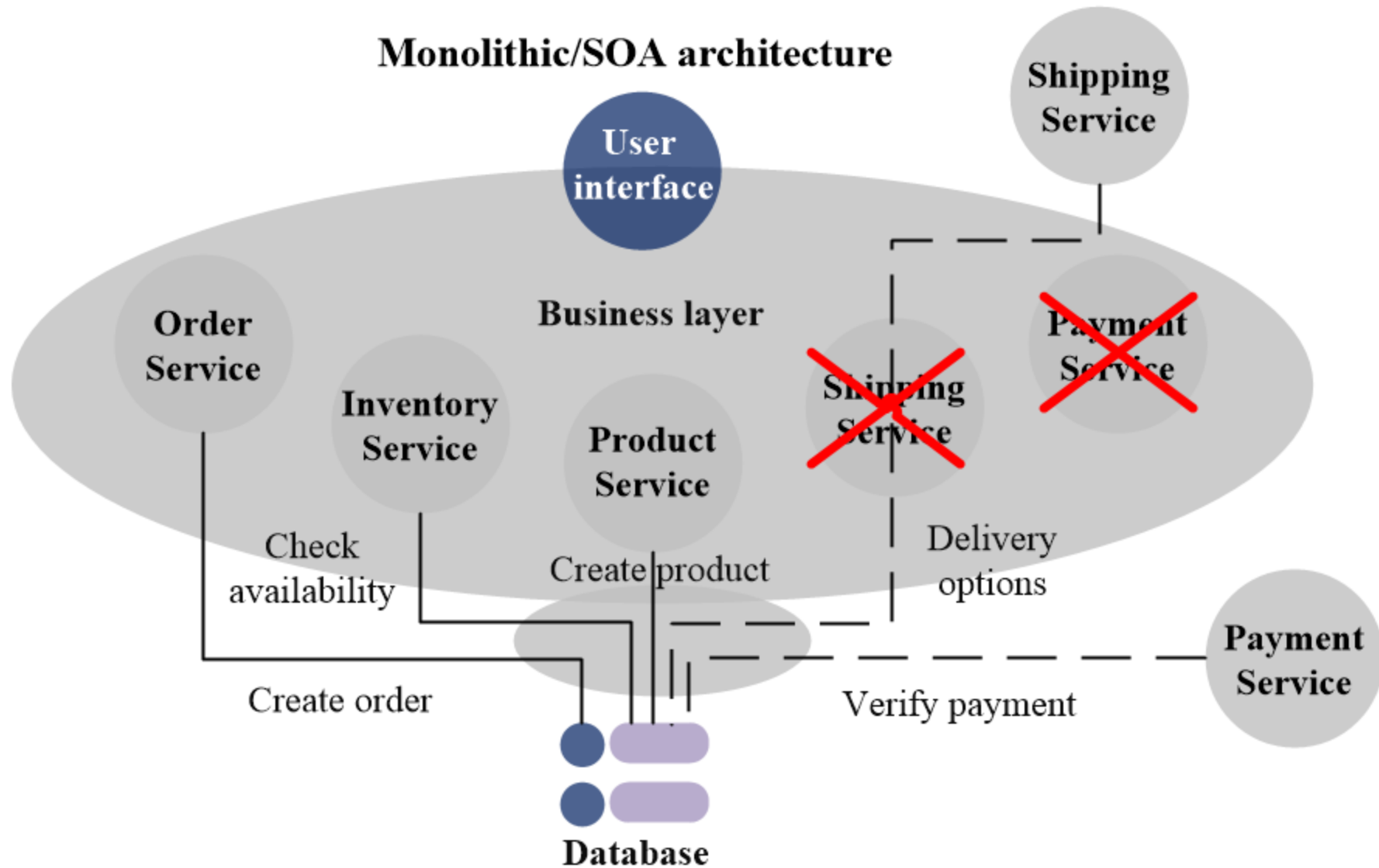


NETFLIX

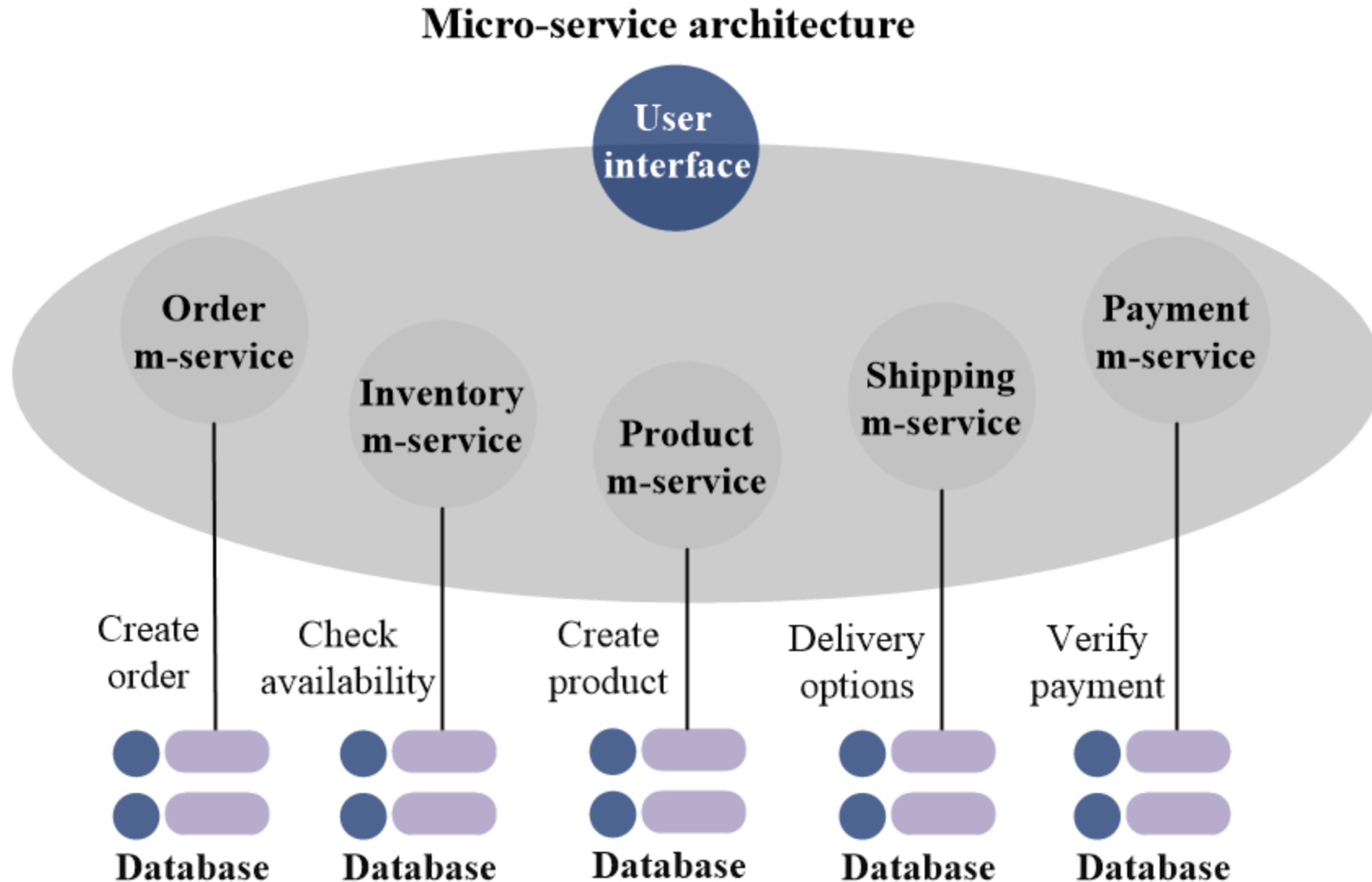


twitter

Back to our ES scenario

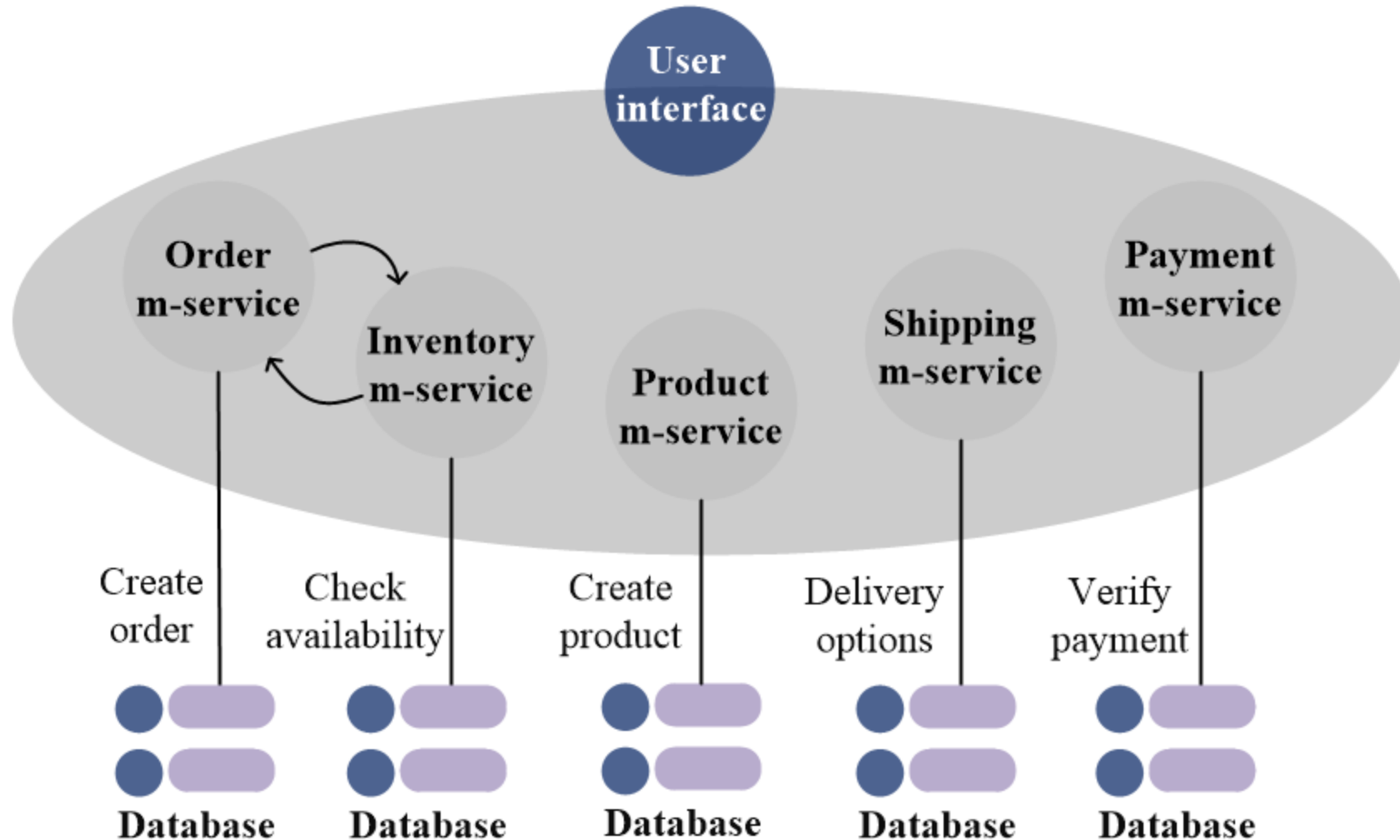


Back to our ES scenario



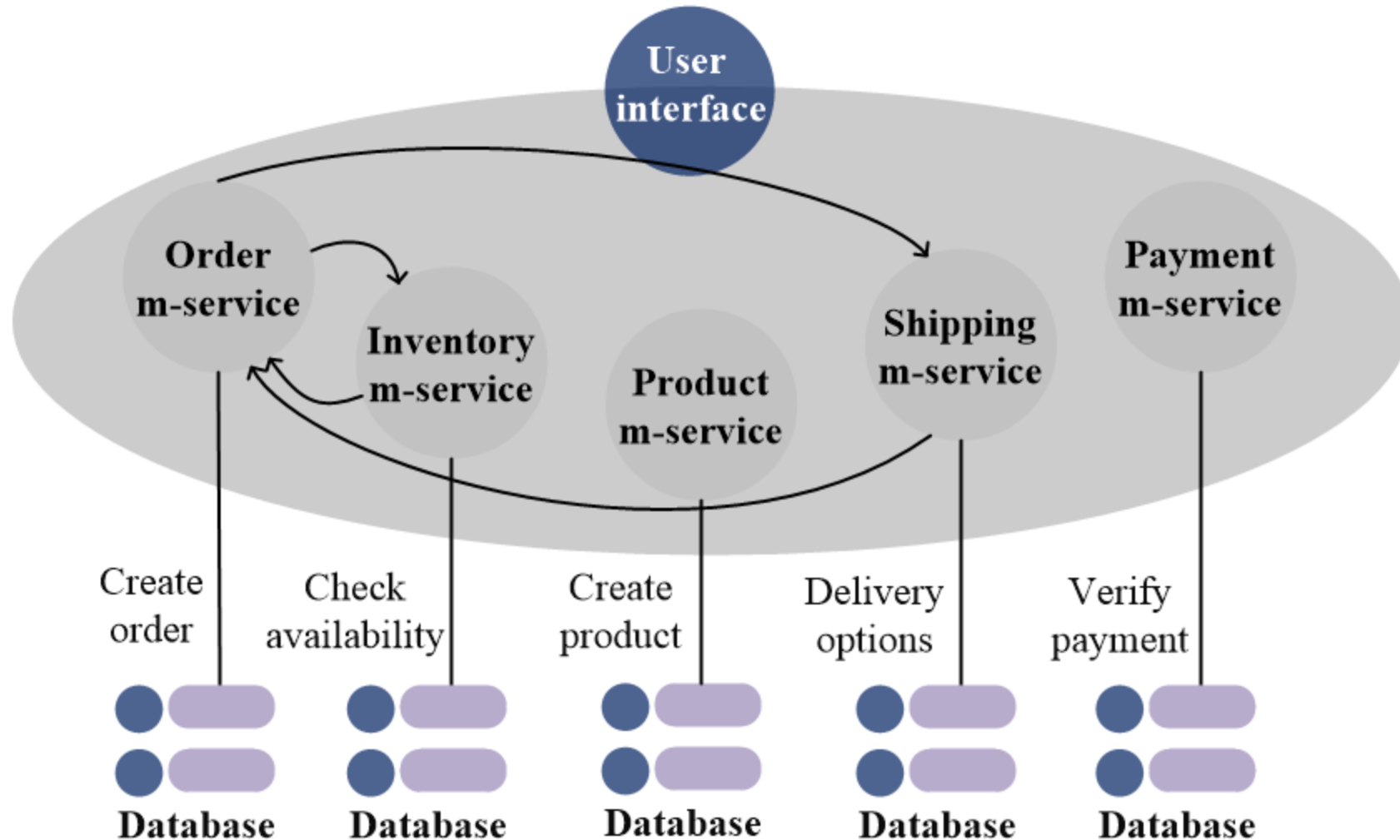
Back to our ES scenario

Micro-service architecture



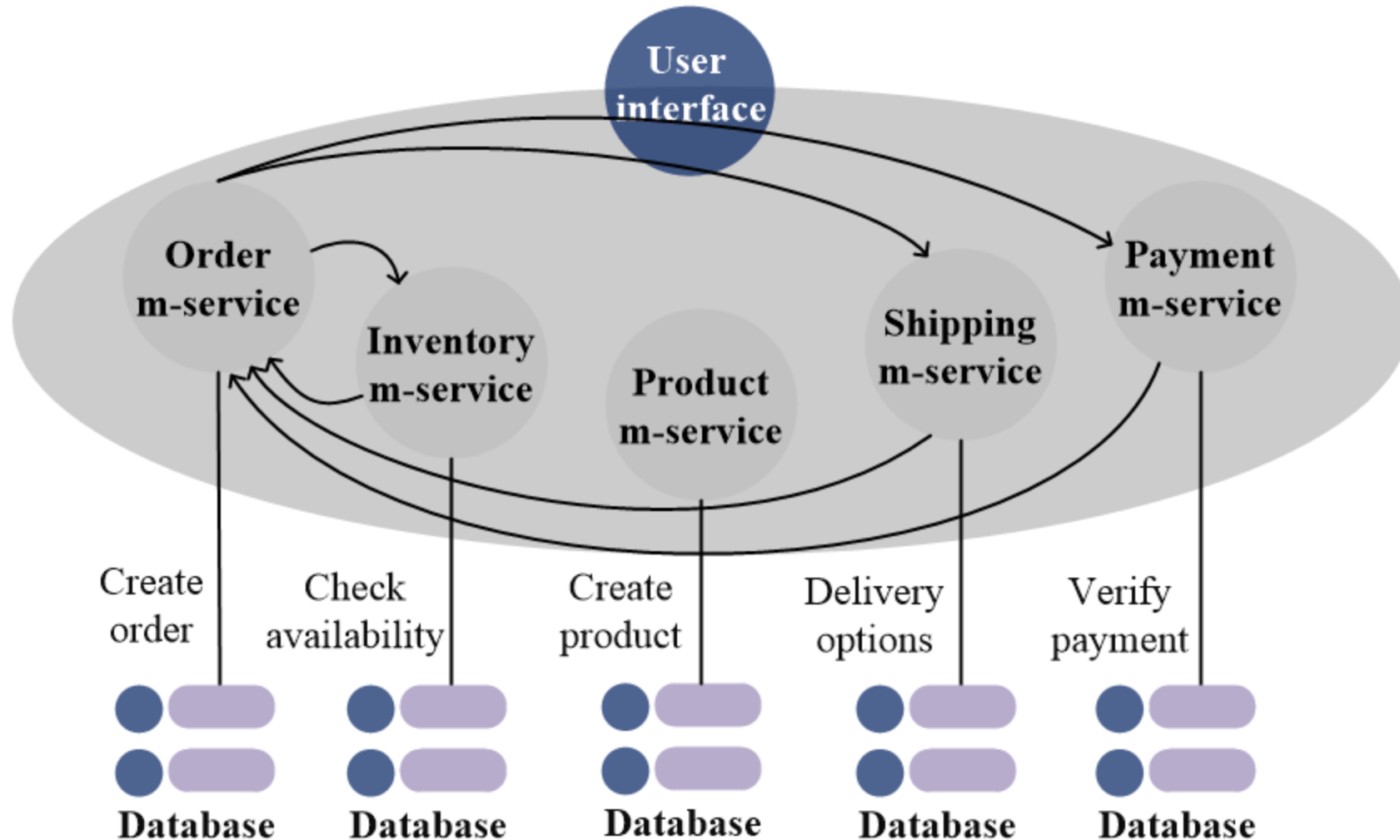
Back to our ES scenario

Micro-service architecture



Back to our ES scenario

Micro-service architecture



Wrap-up and discussion

ESI wrap-up

Old system



New system



ESI wrap-up

Old system



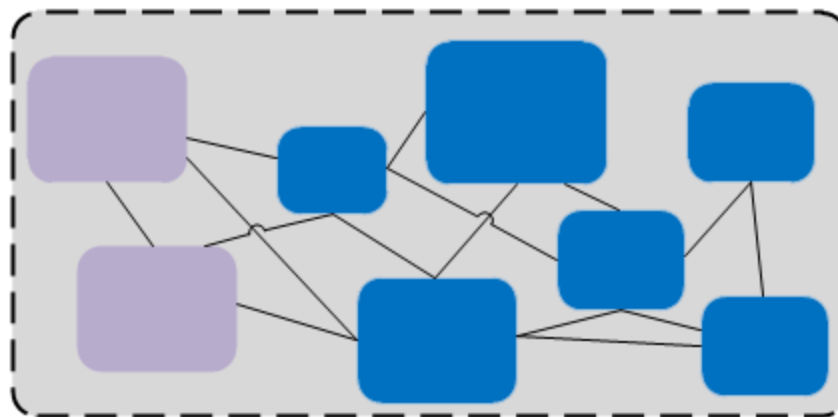
New system



Monolith



Microservice



Monolithic VS Microservices arch.

Discussion:

- What are the key differences between these two architectures?
- What could be the benefits of transforming from **Monolithic** to **micro-services** architecture?
- How can we perform such **transformation**?

Additional resources

- **The Great Migration: from Monolith to Service-Oriented (presentation)**
<https://www.infoq.com/presentations/airbnb-soa-migration/>
- **Monolithic architecture vs microservices**
<https://www.divante.com/blog/monolithic-architecture-vs-microservices>
- **4 Microservices Examples: Amazon, Netflix, Uber, and Etsy**
<https://blog.dreamfactory.com/microservices-examples/>
- **Lessons Learned on Uber's Journey into Microservices (presentation)**
<https://www.infoq.com/presentations/uber-darwin/>

IDE, Tools, etc.

Do it yourself ...

IDE, Tools, etc.



Visual Studio Code

<https://code.visualstudio.com/>

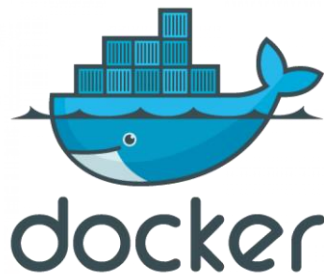


IntelliJ IDEA

<https://www.jetbrains.com/idea/>



Git: <https://git-scm.com/>



Docker Desktop

<https://docs.docker.com/get-docker/>



UNIVERSITY OF TARTU

**Thank You
for your attention**

Mohamad Gharib
mohamad.gharib@ut.ee



unitartu



tartuuniversity

